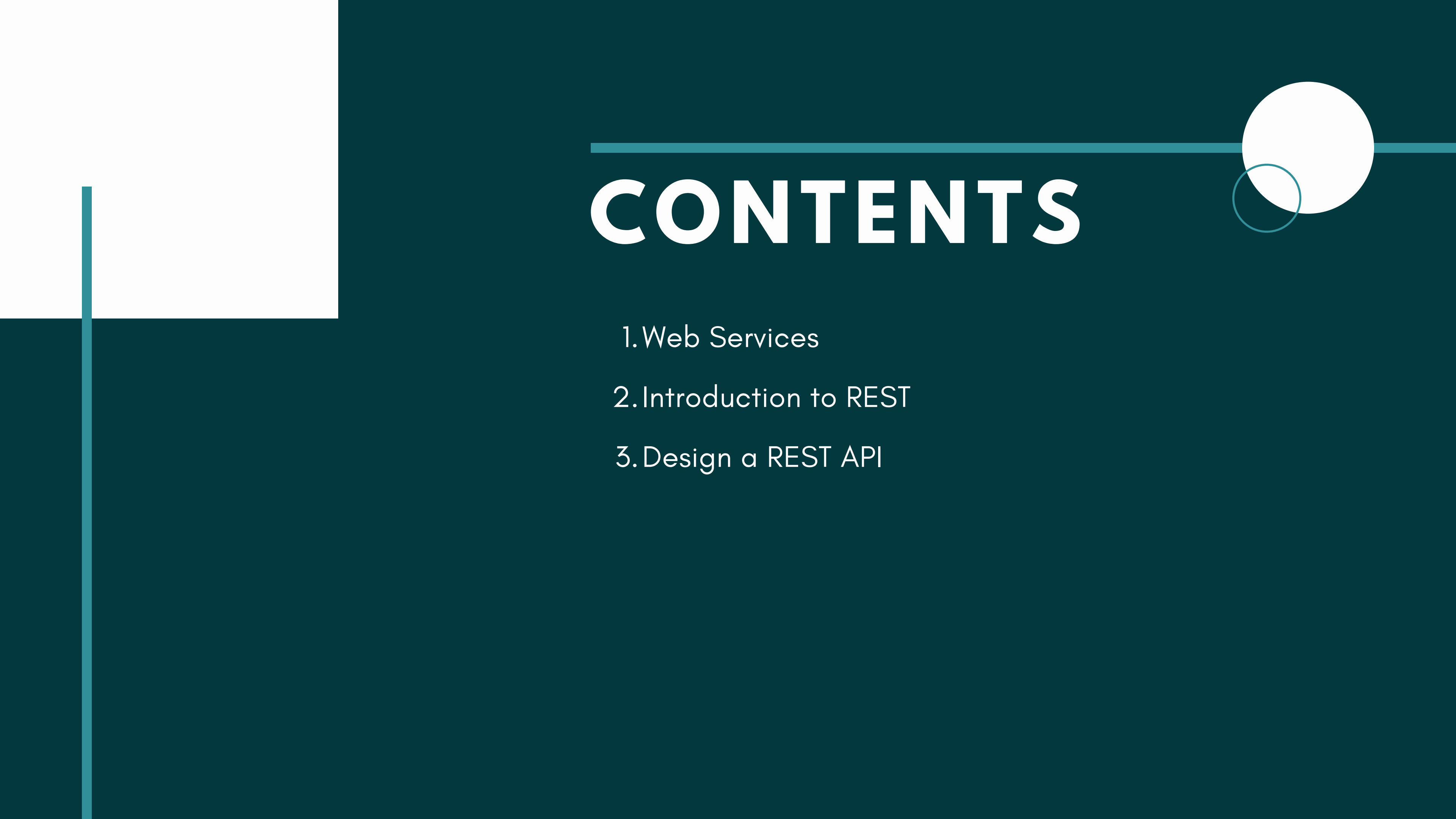




CO3043 – MOBILE DEVELOPMENT

Web Services

MSc. Hoang Le Hai Thanh
High Performance Computing Laboratory – HCMUT



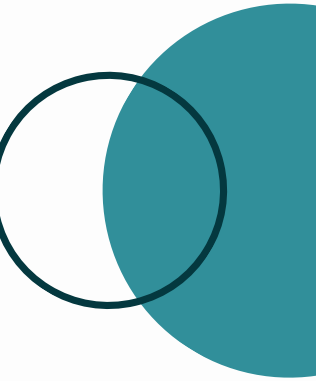
CONTENTS

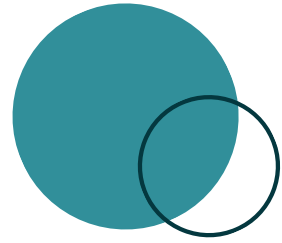
1. Web Services

2. Introduction to REST

3. Design a REST API

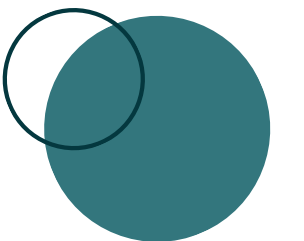
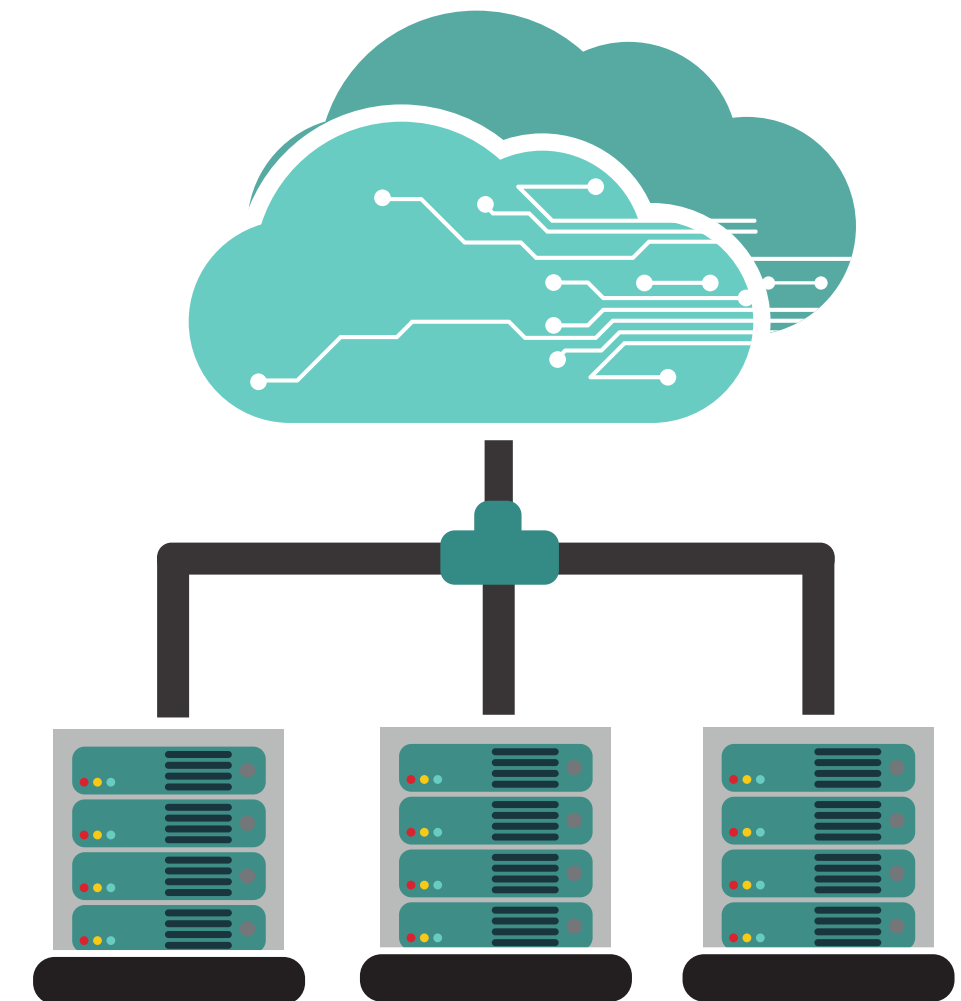
WEB SERVICES

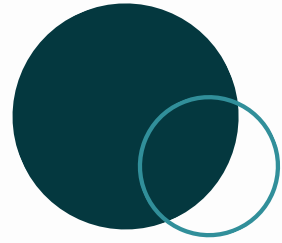




WHAT IS A WEB SERVICE?

- Is available over the Internet or private (intranet) networks
- Uses a standardized XML messaging system
- Is not tied to any one operating system or programming language
- Is self-describing via a common XML grammar
- Is discoverable via a simple find mechanism





WEB SERVICE COMPONENTS

SOAP

Simple Object Access Protocol

SOAP is an XML-based protocol for accessing web services

UDDI

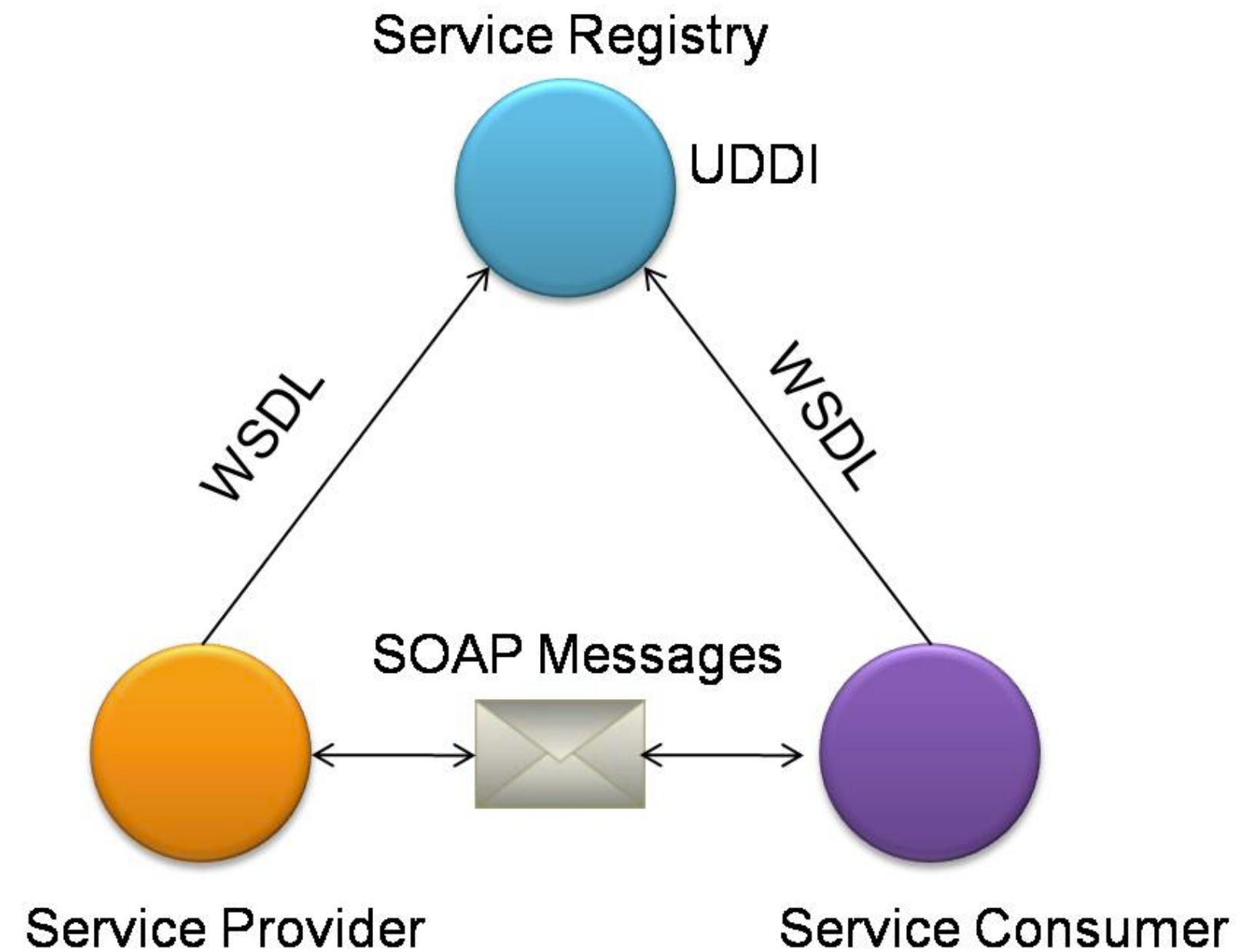
Universal Description, Discovery and Integration.

UDDI is an XML based framework for describing, discovering, and integrating web services.

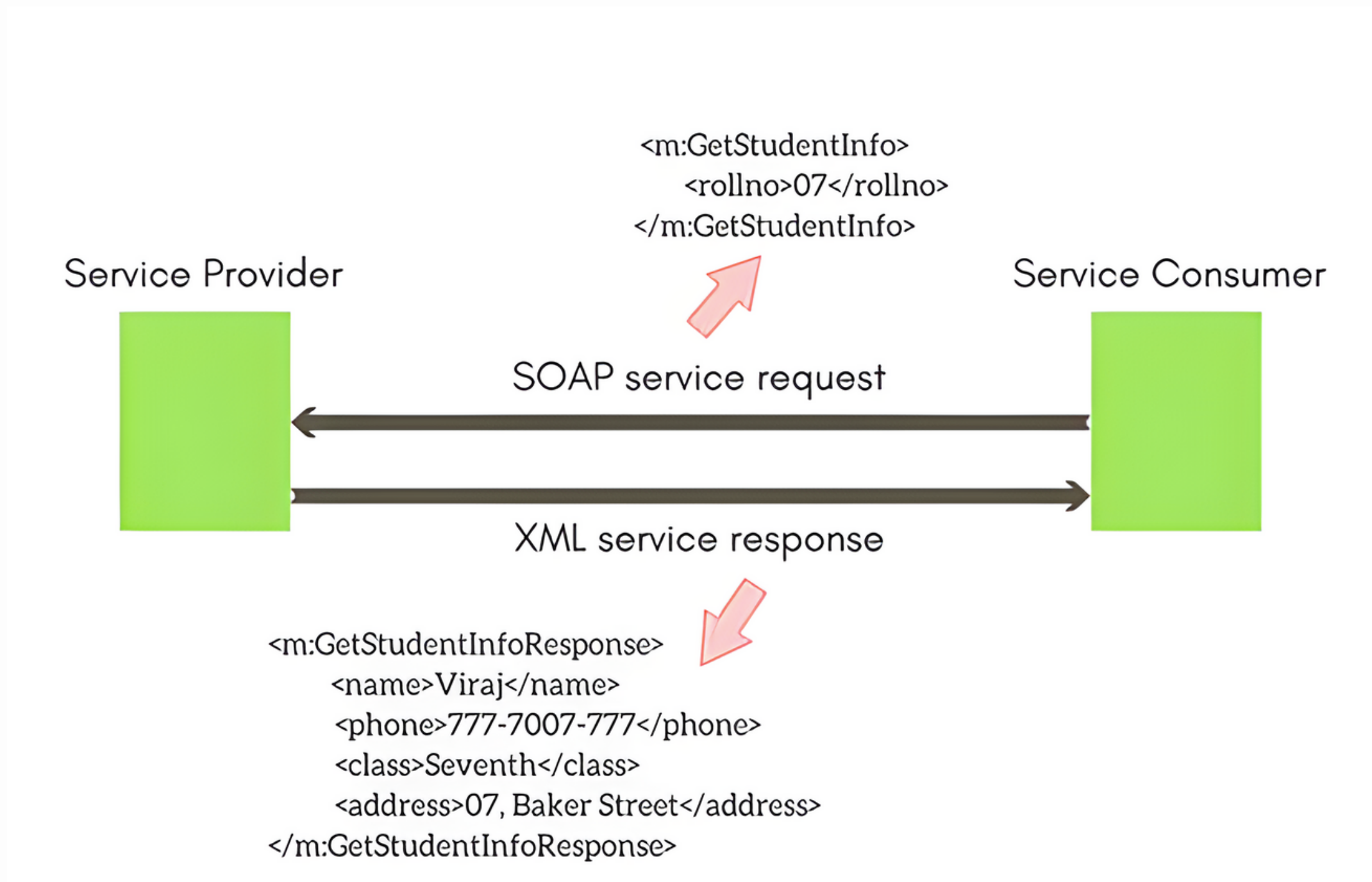
WSDL

Web Services Description Language.

WSDL is an XML document containing information about web services such as method name, method parameter, and how to access it.

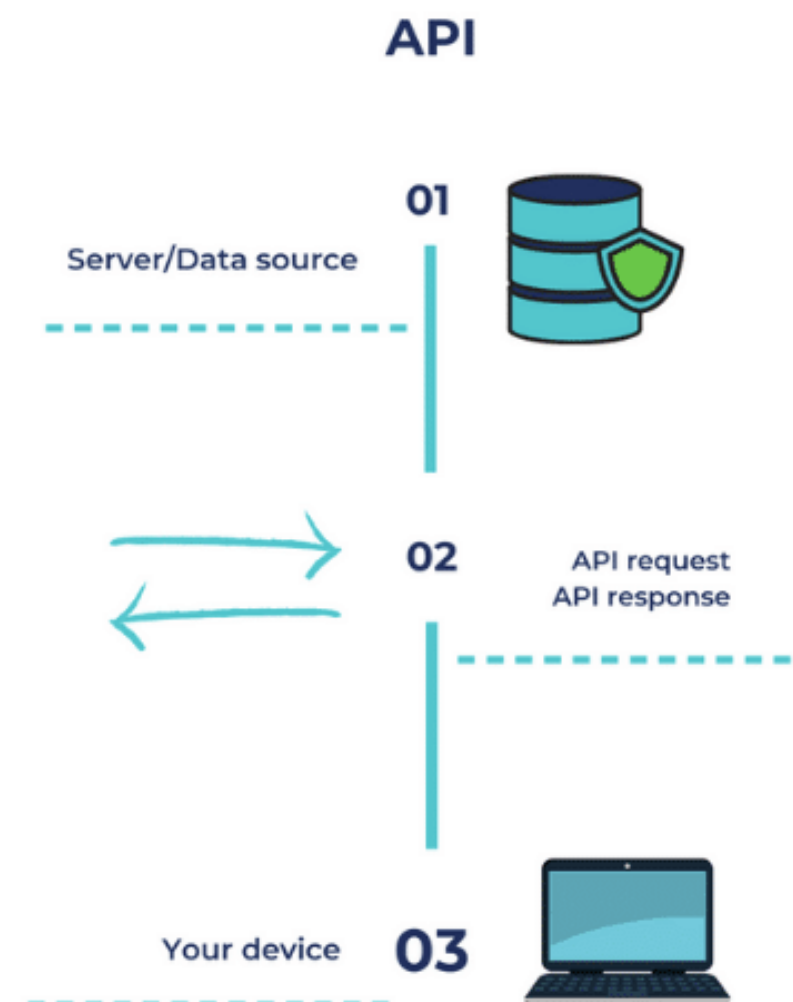
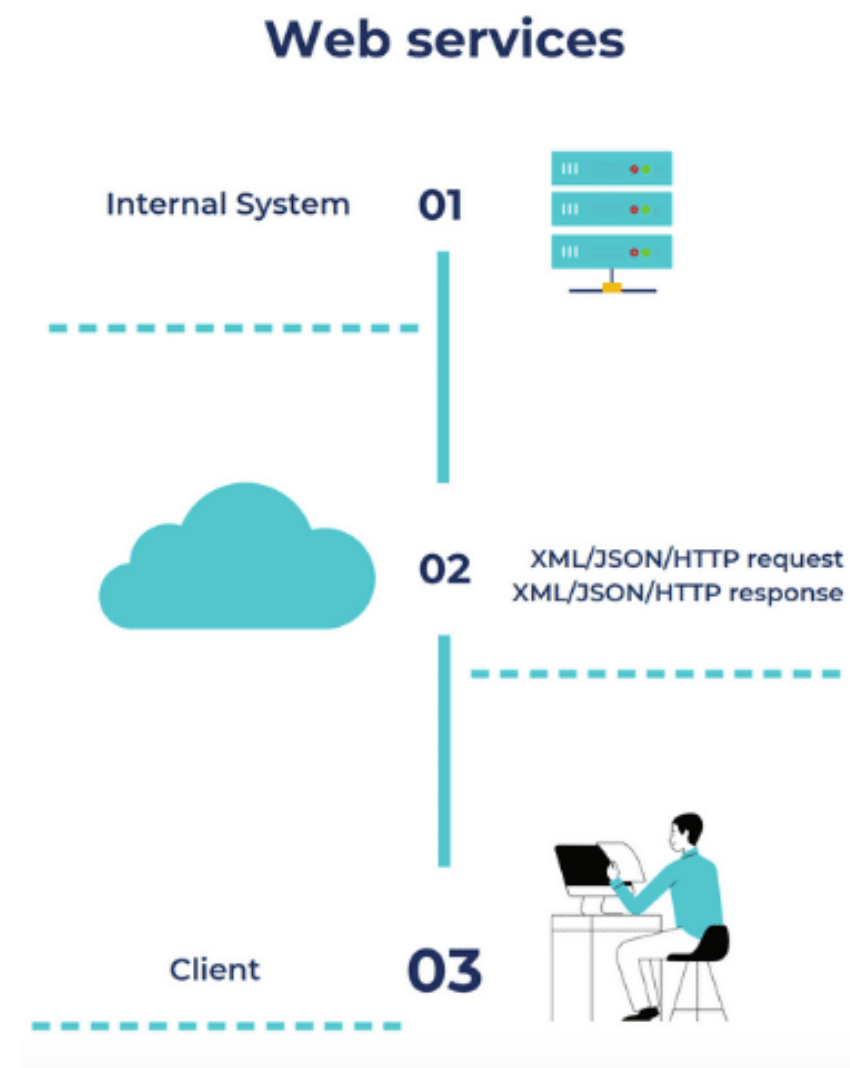


SOAP EXAMPLE



WEB SERVICES

A web service is a resource that is available over the internet



API

An application programming interface (or API for short) is a software component that enables two otherwise unrelated applications to communicate with each other

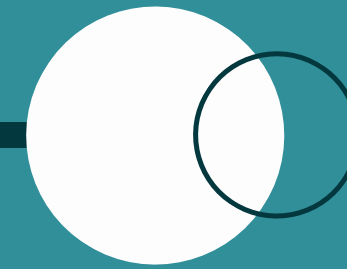
INTRODUCTION TO REST



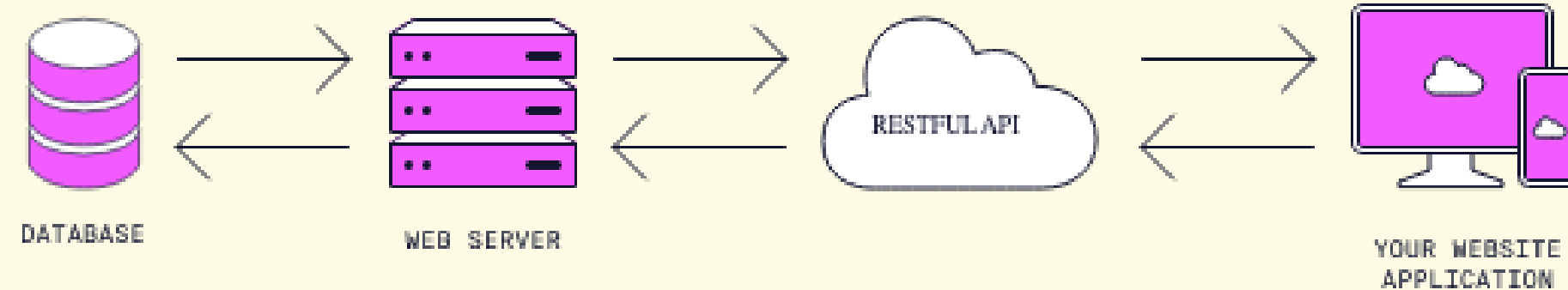
WHAT IS REST?

REST stands for **RE**presentational State Transfer

REST is an architectural style for providing standards between computer systems on the web



What is Rest API?



REST Methods

GET

Use GET requests to only retrieve remote resources

HEAD
OPTIONS
TRACE
CONNECT

POST

Use POST APIs to create new resources

PUT

Use PUT APIs primarily to update an existing resource

PATCH

PATCH only does partial updates

DELETE

As the name applies, DELETE APIs delete the resources

REST CRUD

CREATE

Create



new item



item



item



item



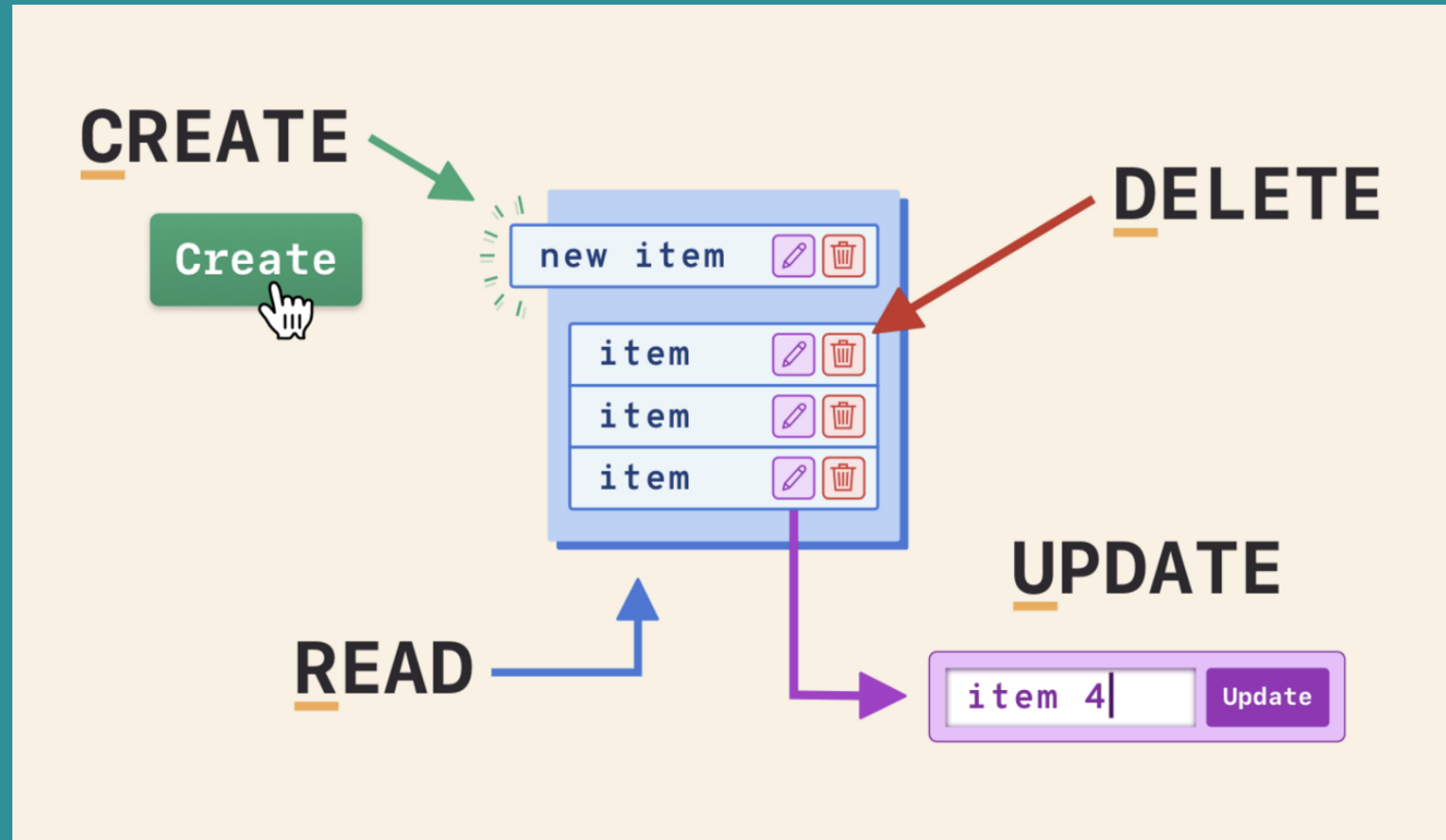
DELETE

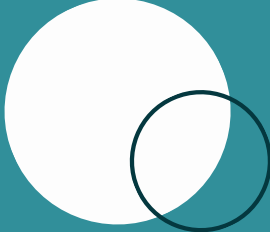
UPDATE

READ

item 4

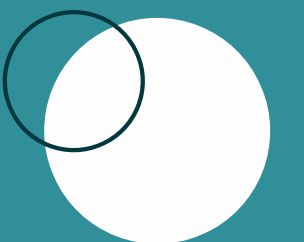
Update



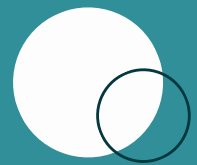


REST Resources

- A resource can be a **singleton** or a **collection**. Ex: **customers** is a collection resource and **customer** is a singleton resource (in a banking domain).
- We can identify the **customers** collection resource using the URI **"/customers"**. We can identify a single **"customer"** resource using the URI **"/customers/{customerId}"**.
- A resource may contain **sub-collection** resources also. Ex: sub-collection resource **"accounts"** of a particular **"customer"** can be identified using the URN **"/customers/{customerId}/accounts"**

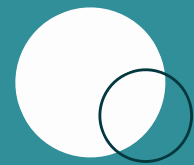


RESTful Guidelines



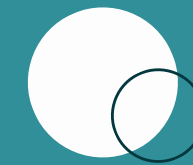
CLIENT-SERVER

Client applications and server applications **MUST** be able to evolve separately without any dependency on each other.



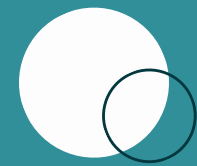
STATELESS

The server will not store anything about the latest HTTP request the client made. It will treat every request as new. No session, no history.



UNIFORM INTERFACE

All resources should be accessible through a common approach such as HTTP GET and similarly modified using a consistent approach.



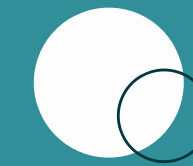
CACHEABLE

Caching shall be applied to resources when applicable, and then these resources **MUST** declare themselves cacheable.
Caching can be implemented on the server or client-side.



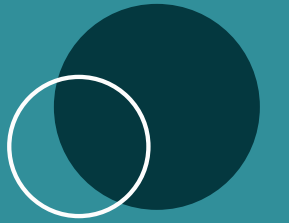
LAYERED SYSTEM

Each layer doesn't know any thing about any layer other than that of immediate layer and there can be lot of intermediate servers between client and the end server.



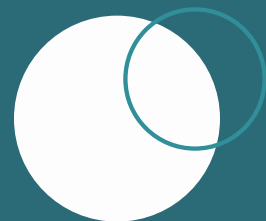
CODE ON DEMAND (OPTIONAL)

REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts.



SOAP

- SOAP is a protocol
- SOAP cannot make use of REST
- SOAP uses service interfaces to expose its functionality to client applications
- SOAP requires more bandwidth for its presentation
- SOAP can only work with XML format. As seen from SOAP messages, all data passed is in XML format.
- Support Stateful operations and security mechanisms



REST

- REST is an architectural style
- REST can make use of SOAP
- REST uses Uniform Service locators to access the components on the hardware device.
- REST messages is lightweight
- REST permits different data formats such as Plain text, HTML, XML, JSON, etc.
- Support caching



DESIGN A REST API



Idempotent REST APIs



- **Idempotency** is the ability for an operation to be carried out multiple times without changing the state of the system.
- An idempotent HTTP method is a method that can be invoked many times without different outcomes.
- It should not matter if the method has been called only once, or ten times over. The result should always be the same.

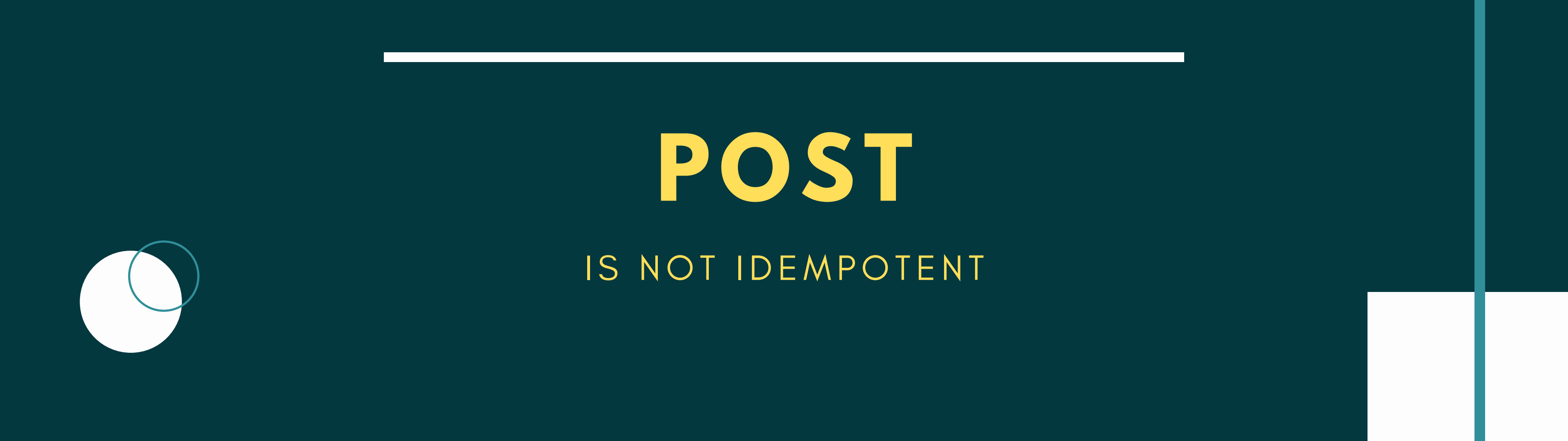


GET PUT DELETE PATCH

ARE IDEMPOTENT

POST

IS NOT IDEMPOTENT



REST Resources

- A resource can be a **singleton** or a **collection**. Ex: **customers** is a collection resource and **customer** is a singleton resource (in a banking domain).
- We can identify the **customers** collection resource using the URI **"/customers"**. We can identify a single **"customer"** resource using the URI **"/customers/{customerId}"**.
- A resource may contain **sub-collection** resources also. Ex: sub-collection resource **"accounts"** of a particular **"customer"** can be identified using the URN **"/customers/{customerId}/accounts"**

REST Best practices

Use nouns to represent resources

RESTful URI should refer to a resource that is a thing (noun) instead of referring to an action (verb) because nouns have properties that verbs do not have – similarly, resources have attributes

`http://api.example.com/device-management/managed-devices`

`http://api.example.com/device-management/managed-devices/{device-id}`

`http://api.example.com/user-management/users`

Use Nesting on Endpoints to Show Relationships

Ex: A blog post might have their comments, so to retrieve these comments, an endpoint like

`https://mysite.com/posts/{postId}/comments` would make sense.

REST Best practices

Use Nouns Instead of Verbs in Endpoints

Because HTTP methods such as GET, POST, PUT, PATCH, and DELETE are already in verb form for performing basic CRUD (Create, Read, Update, Delete) operations, the endpoints should use nouns, signifying what each of them does.

Use JSON as the Format for Sending and Receiving Data

To ensure the client interprets JSON data correctly, you should set the Content-Type type in the response header to application/json while making the request.

REST Best practices

Use Filtering, Sorting, and Pagination to Retrieve the Data Requested

GET requests over collection resources can potentially return a large number of items. You should design a web API to limit the amount of data returned by any single request.

Ex: `/orders?limit=25&offset=50`

Be Clear with Versioning

REST APIs should have different versions, so you don't force clients (users) to migrate to new versions. This might even break the application if you're not careful.

Many RESTful APIs from tech giants and individuals usually comes like this:

`https://mysite.com/v1/` for version 1

`https://mysite.com/v2` for version 2

REST Best practices

Handle errors gracefully and return standard error codes

To eliminate confusion for API users when an error occurs, we should handle errors gracefully and return HTTP response codes that indicate what kind of error occurred.

Common error HTTP status codes include:

- **400 Bad Request** – This means that client-side input fails validation.
- **401 Unauthorized** – This means the user isn't not authorized to access a resource. It usually returns when the user isn't authenticated.
- **403 Forbidden** – This means the user is authenticated, but it's not allowed to access a resource.
- **404 Not Found** – This indicates that a resource is not found.
- **500 Internal server error** – This is a generic server error. It probably shouldn't be thrown explicitly.
- **502 Bad Gateway** – This indicates an invalid response from an upstream server.
- **503 Service Unavailable** – This indicates that something unexpected happened on server side (It can be anything like server overload, some parts of the system failed, etc.).

REST Best practices

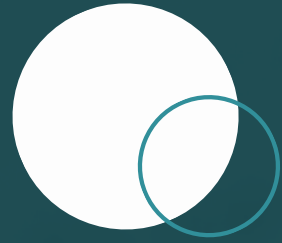
Cache data to improve performance

We can add caching to return data from the local memory cache instead of querying the database to get the data every time we want to retrieve some data that users request. The good thing about caching is that users can get data faster. However, the data that users get may be outdated.

Provide Accurate API Documentation

When you make a REST API, you need to help clients (consumers) learn and figure out how to use it correctly. The best way to do this is by providing good documentation for the API.

One of the most common tools you can use for API documentation is **Swagger**. And you can also use **Postman**, one of the most common API testing tools in software development, to document your APIs.



Any question?

MSc. Hoang Le Hai Thanh

High Performance Computing Laboratory - HCMUT

thanhhoang@hcmut.edu.vn